VLSI ELECTRONICS: MICROSTRUCTURE SCIENCE, VOL. 3

Chapter 2

VLSI Design and Design Aid Requirements

CHARLES HOUSE
KENT HARDAGE

Hewlett-Packard Company
Colorado Springs, Colorado

I. INTRODUCTION

The Hewlett-Packard (HP) Company has had considerable experience with custom integrated-circuit design, fabrication, and manufacture. For several reasons, the experience within the company is not necessarily correlated with that of other large users of integrated circuits. In this chapter we shall explore some of the distinctions, the reasons for them, and some conclusions that may be drawn.

First, it is important to understand that Hewlett-Packard is both a large user of commercially available integrated circuits and a major producer of captive integrated circuits. During the past five years, the company has regularly purchased about 10% of the total memory devices produced by

ISBN 0-12-234103-1

commercial vendors, and it perennially ranks in the top ten electronics companies in total integrated-circuit purchases by dollar volume on a worldwide basis.

The company has at least a dozen integrated-circuit fabrication facilities, ranking it among the top five electronics companies in terms of total semiconductor manufacture for captive or proprietary usage. It is also an industry supplier of a selected few components, such as silicon high-frequency devices (10–40 GHz), gallium arsenide high-frequency devices (40–80 GHz), and gallium arsenide phosphide light-emitting diodes. The range of technologies that the company has in production include bipolar TTL and ECL, NMOS, CMOS, HMOS, and SOS, as well as the gallium-arsenide-based products.

Industry observers often note the relatively low wafer start rate of HP's IC shops and decry the low efficiency of capital investment. While this is true, the approach is modeled on the company's unique decentralization of engineering R&D teams in geographically dispersed areas. Such decentralization is so complete that it remains true, as it has for 20 years, that HP's divisions and design teams are roughly an order of magnitude smaller than those of virtually any other equivalent company in the electronics industry [1]. The premise, of course, is that such decentralization yields a higher productivity in terms of motivation, incentive, understanding, and cooperation in the design tasks required to retain state-of-the-art leadership. Whether such gains offset the cost of capital investment is conjecture, but it is a major distinction in the comparative approach of the company.

Given the preceding points, we may anticipate that some differences in approach and viewpoint that may be available within the company may be of interest for the industry at large as the era of relatively low-cost, high-density, custom integrated circuits dawns. In particular, in this chapter we shall focus on quick access to "workhorse" technologies, with attention being placed on "comprehensive" design aids for the entire design process as the more typical and useful contribution for designers seeking to utilize the industry's increasing sophistication in integrated circuits.

II. STATE-OF-THE-ART VERSUS WORKHORSE TECHNOLOGIES

Much of the work within Hewlett-Packard parallels that of the industry in terms of a fundamental need to have process and product development at the forefront of technology. This is a major reason for captive integrated-circuit facilities within the company, and it has been very successful as a strategy for many product areas.

The company has been a pioneer in silicon-on-sapphire processing, reducing it to high-volume production processes for major computer products before concentrating on the radiation-hardness qualities of the technology. It pioneered in high-frequency gallium-arsenide work for communications equipment in the 40–80-GHz frequencies. It has concentrated heavily on the broadband light spectrum characteristics of light-emitting diodes and arrays. It produced and delivered many thousands of 16-bit microcomputers in an NMOS chipset two years before the Intel 8086 was available. It will probably market the first minicomputer line based on a proprietary 32-bit microcomputer, again before the commercial semiconductor manufacturers deliver many 32-bit microcomputers for evaluation purposes [2].

The problems and solutions inherent in this end of HP's VLSI design needs are not that different from those described elsewhere in this book. Submicron geometry fabrication concerns, process refinement and innovation, and materials technology are all crucial, and numerous creative ways of meeting the needs are tried and modified in order to succeed. As one might expect, the gains are often due more to effective brute force than to really clever innovation, although there is plenty of room for contributions in all areas.

Another place where the company has been able to make some contributions of a unique nature is in the workhorse technologies, and this is largely a contribution that has been ignored in the literature in general. Because of the pervasive nature of custom integrated-circuit design possibilities in the next ten years, this may be a fruitful area to explore in some detail. It is very possible that the way that most engineers will experience the benefits and limitations of VLSI processing power will be via an experience somewhat akin to the current HP workhorse experience.

What are the concerns and goals of designers using HP workhorse technologies? Quickly stated, they include

(1) low-volume irregular structures;
(2) low development time and cost, coupled with moderate chip cost;
(3) ease and convenience of design;
(4) good simulation capability;
(5) wide parametric variation;
(6) good testability and consistent yield;
(7) repeatability at a second process shop; and
(8) design access to a remote process shop.

It does not take much consideration to realize that most commercial semiconductor manufacturers are not especially interested in serving these needs, nor are they effectively able to do so. Nor does it take much thought to realize that HP's diverse instrumentation product line has given rise to most of these needs, and they are not easily served by relying on one or two technologies, since an enormous range of frequencies, impedances, dynamic ranges, noise immunities, and other parametric concerns must be considered in the real-time measurement world.

What may not be so obvious is the similarity between HP's needs today and those of the generalized designer for tomorrow, when such obvious needs as integrated real-time process control in all sectors of society will require custom designs built around the preceding requirements.

Let us examine each of the preceding points in some detail and then attempt to draft some conclusions for the VLSI requirements that will be coming. Following that, we shall examine a specific program currently being developed in the Logic Systems Operation for its utilization and results with these concepts. Finally, we shall close with a brief description of a possible model for companies interested in developing a "design aids" center such as described herein.

III. DESIGNER GOALS IN WORKHORSE TECHNOLOGIES

The goals listed in Section II for HP designers using workhorse technologies have been evolving for about a decade. Today there are some four or five bipolar processes available throughout the company, with TTL, Schottky, ECL, EFL, and I^2L all characterized for each process. The processes may be structured for low noise and very low propagation delays for oscilloscope and signal analyzer preamplifiers, or they may feature wide-dynamic-range line-driver and output-stage capability. Some processes routinely can deliver 12-GHz devices, but it is more typical to talk in terms of 2- to 5-GHz devices and 1000 to 2500 devices per chip. Several MOS processes also exist, using NMOS technology and 3–4-μm geometries, which provide good but not first-rate technology.

a. Low-Volume Irregular Structures. At the outset, it is important to realize that HP products may use about 100 ICs on average, with perhaps ten of them custom. A typical product will have about 5 to 10% of the total selling price in the direct material of the ICs (usually about one-half in the custom circuits). Moreover, a typical product will have sales in the 50–100/month range. Thus, when we talk of low volume, we really mean about 10,000 custom chips/yr for each product and that usually is spread among four to six different chip designs. Based on a typical five-year product life, this results in, say, five custom designs, each of which requires between 5000 and 10,000 total chip utilization.

b. Low Development Time and Cost. Now it becomes obvious why low design time and low development cost are rather more important than the chip cost. Working with a typical $100K/yr per designer plus a $50K/design set of mask and wafer costs, it is not difficult to spend $250K/chip for development. This amortizes to $25 to $50/chip in the completed instrument, in addition to the direct material cost of the tested and packaged chip itself. Frequently, for HP designs that are "quasi" state of the art in workhorse technologies, it has been typical to incur costs substantially higher than these. It is also quite possible in the more esoteric instrumentation areas to find sales volumes in the 25/month range. This combination can give a deadly $100–$200 development surcharge/chip that is hard to recover unless the measurement contribution is significant.

c. Good Simulation Capability. Effective simulation, à la SPICE, has been one of the historic approaches to better design efficiency. To the extent that the circuits and devices are characterized and modeled correctly and the circuit simulation gives results that the process can later fulfill, the design confidence goes up and the number of iterations goes down. The failings of simulation in our view primarily occur in larger cell definitions, where cumulative errors cause significant noncorrelation and repeated characterization attempts must be made in order to develop a library of structured cells that are accurately depicted.

d. Wide Parametric Variation. Wide parametric requirements doubly burden the process, in that they are harder to model effectively and they often place constraints on the processing team wherein iterative steps in wafer processing are made in different degrees and directions for different characteristics. For low-volume shops, this affects productivity, for nothing is ever really done on a flow basis—every batch is custom processed, with the attendant increased possibility for error via nonroutine handling.

e. Good Testability. Consequently, the testing process is taxed as well, for large, well-designed testers aimed at throughput for a specific class of circuits are out of the question. Moreover, the odds that specific dynamic parameters are the major ones of interest ensures that the testability concerns are high in order to save the costs of going on to packaging in order to determine the quality of the chips. Our experience suggests that several unspecified parameters are usually the most critical ones anyway. An example we encountered had to do with the power supply distribution on-board a chip of 160 mils/side at frequencies above 150 MHz.

No one disagrees that the original circuit designer must be intimately involved in developing effective testing models, but, ironically and unfortunately, the designer working with a workhorse technology is inclined to think that such a task is less vital than a designer working with a forefront technology.

f. Repeatability. From a design security standpoint, HP designers have long since adopted the requirement that virtually all parts in a major product have second-sourcing capability, to guard against any number of disabling events. This is a difficult thing to assure with custom proprietary ICs, yet it is a major requirement that is usually addressed. Consequently, there is a need to track several of the preceding concerns through two shops, either simultaneously or sequentially. Both require full process and design characterization, along with concomitant testing, and it imposes some rigid boundary conditions on "stretching" the capabilities of a local process. Thus, this requirement has the beneficial effect of stabilizing a workhorse technology, albeit at the risk of achieving some specific marginal improvements in given designs.

g. Design Access. This shows up more impressively in the requirement that IC shops allow, even encourage, remote designer utilization. The idea behind this concept at HP is twofold: (1) there will never be an IC shop available at each dispersed division; and (2) if there were, it could not possibly involve the entire range of technologies that that division's designers would like to incorporate into custom designs. As a corollary, this concept does allow some increased capital equipment utilization. Anyone who has tried to work with remote commercial silicon suppliers on custom designs appreciates that success is fully dependent on a match in technologies, a superb documentation path, and many coordinated discussions at critical junctures. The commercial semiconductor industry, as well as HP, has yet to solve this problem effectively. Gate arrays have helped, but design aids and a design interface defined at a higher description level may help even more.

IV. NEXT-GENERATION REQUIREMENTS

Does the next level of sophistication in semiconductor technology—VLSI—represent an evolution or a revolution? Will our strategies for design and analysis be capable of further refinement or is it time to reassess our entire process? Given that we must make a choice for one approach versus another, what implications does this choice hold for the potential of VLSI technology in diverse applications throughout our society?

When we examine the complexity and capability of VLSI technology, it is really quite extraordinary. Moreover, it quickly becomes obvious that present-day design techniques are woefully inadequate for the challenges that VLSI presents. To be sure, many VLSI designs will be accomplished with brute-force extensions of present-day approaches, but they will be limited to those applications that are high-volume, "regular" (in the partitioning sense), and extensions of the complexity of existing architectures, such as 8- to 16- to 32-bit CPUs or 4K- to 16K- to 64K- to 1-M bit or byte memories. Such applications occur in computational and communications environments and, perhaps, in consumer product areas such as automotive collision-avoidance systems. Unfortunately, many exciting potential areas such as integrated real-time process control are virtually excluded because of the unique nature of most applications.

In order to realize the potential that VLSI technology offers, it is imperative that we develop design aids that really *help* in the design and analysis of systems that utilize VLSI's inherent complexity and power. Four major changes are required to achieve this goal. They are (1) high-level language descriptions for both hardware and software, along with a blending of their optimization and a greatly increased "user-friendliness" factor for the tools; (2) easy, accurate simulations of the resultant designs, complete with benchmarking performance measures for effective comparative analysis; (3) linked design and analysis techniques and data files so that many team members, both within the professional design areas and in functional support areas, can share developed information; and (4) easy access to silicon VLSI processing centers for low-cost, low-volume, fast-turnaround, complex chip sets. Does this sound too good to be true? It is not, or at least it should not, and today is not too soon for us to begin to work in this direction.

V. THE PROBLEM AND THE APPROACH

The impetus for a change in design methodology occurred at the Logic Systems Operation in mid-1978 at the conclusion of a major program to update a product line from manual switch-controlled random logic TTL circuit architectures to programmable keyboard-interactive microprocessor-controlled ECL circuit architectures. A typical product in this newer line included 300 14-pin DIP ICs, plus 20–30 LSI chips for CPU and memory, dissipating 200 W/ft^3 of total box space (Fig. 1).

The next generation of equipment, due out in 1982–1983, was postulated to need an improvement of 10 to 20 times in complexity along with an increase in speed of 5 to 10 times and a gain in data acquisition memory

Fig. 1. HP1610A logic state analyzer.

of 25 to 100 times. Moreover, it was felt that there was a need for much more permutation capability for customer selection of feature sets. Custom integration looked like the only feasible recourse for many of the machine characteristics.

Fortunately, the local captive IC shop was a bipolar facility with both 1-GHz and 2-GHz device capabilities. Gate delays were in the 1–3-nsec range and a gate complexity of 750 gates/chip was realizable on a 150-mil square die with power dissipation of approximately 1–2 W. A small number of chips of comparable complexity were designed in the process. Unfortunately, the typical design process was extremely lengthy, as illustrated in Fig. 2. One of the chief impediments in the design process was the fact that designs were implemented at the same level that the artwork was generated. The engineer laid out each transistor geometry using rectangles to determine transistor characteristics.

In addition to the extremely long development time, few analysis tools were used and the possibility of a correct design on first cut was very low. Recuts would extend the development time. These designers were truly "chewing on the bark incessantly, while trying to find the forest." To base a future-generation product line, which appeared unrealizable

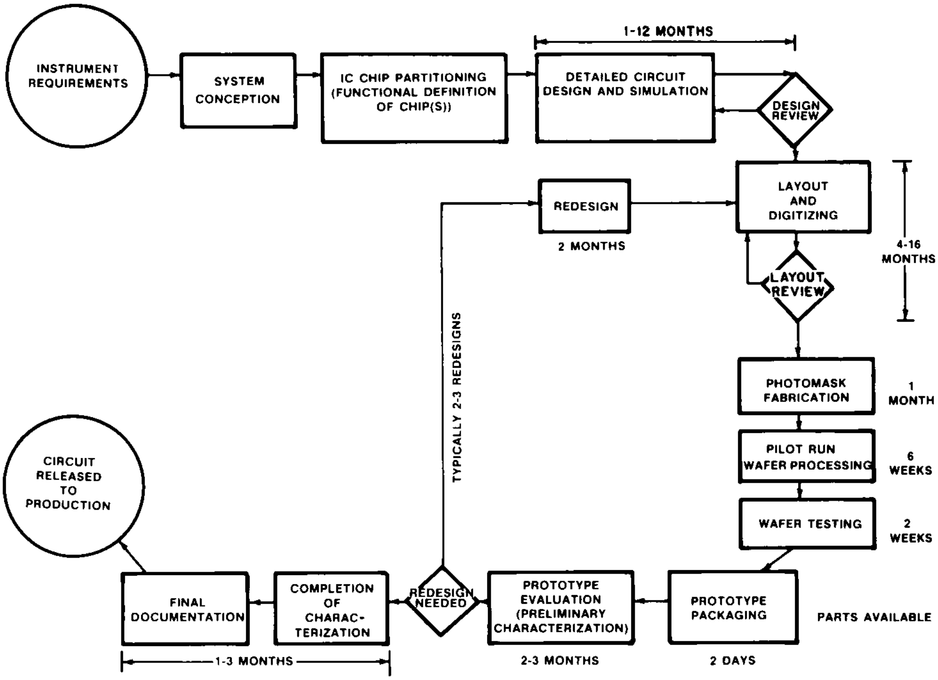


Fig. 2. Conventional custom IC design cycle.

without a number of custom ICs, on such a design approach seemed risky at best. It would consume an enormous amount of engineering resources. Would the product fit the market place for which it was designed with such a long development time?

A task force of engineering managers in the product development laboratory was formed to meet for several weeks with the silicon process managers in a seminar. Each team became aware of the other's capabilities, approaches, and problems. At the conclusion of this information transfer, the instrument laboratory made a request for a substantially changed approach to IC design as follows:

(1) A high-level set of "structured cells" would be developed and characterized for use in digital bipolar designs.

(2) The complete design cycle would be addressed, with emphasis placed on much shorter artwork generation time.

(3) Extensive analysis and design verification tools would be used to guarantee correct designs on first cut.

(4) System designers without detailed knowledge of integrated-device physics would be able to specify IC designs.

A. Project ASAP

An integrated-circuit design team was formed and a new methodology, which has become known as ASAP (Advanced Symbolic Artwork Preparation), was the result. This methodology has created a new design strategy for custom ICs in the HP Logic Systems Operation. This has now been extended to HP's general IC tools at other divisions. To address the four previously stated objectives, ASAP uses techniques for LSI chip development that parallel software development methodologies. Figure 3 illustrates that parallel.

Software methodologies manage complexity by using macrodescriptions, high-level languages, and structured programming approaches. The ASAP design system was developed to offer the same set of capabilities to designers of custom integrated circuits. It does this by extending the software concept of mapping a program into a one-dimensional array of memory to the mapping of a graphic symbolic hardware description into the 2.5-dimensional physical plane of IC masks. Thus, ASAP enbles the designer and the computer to use symbolic abstractions for structural, physical, and some behavioral properties. Completed designs can be verified for correctness before they are assembled into IC mask descriptions for fabrication. Multiple representations, coupled with a hierarchical design approach using regular structures and global wiring management, have been very effective in custom IC design.

Conventional computer-aided design (CAD) systems use graphic design tools to capture, edit, and output the detailed physical description of the IC mask. However, while artwork generation always attracts the most attention, the complete design cycle must be addressed for the design philosophy to be effective. While a graphic interface between designer and data is convenient and useful, manipulating the design at the mask level is very limiting. The ASAP system uses a graphic design description but modifies and extends the data representation in the following ways:

(1) Superfluous physical information is reduced by using high-level symbolic representations of components and interconnects.

(2) With this unnecessary information deleted, the designer can see more of the design at a smaller scale, thus gaining a better overall perspective.

(3) Structural and behavior descriptions have been added to aid design verification. The structural description defines the logical interconnections of the system, while the behavioral description provides basic information about interconnection points (for example, whether they are inputs or outputs and how many loads they will drive).

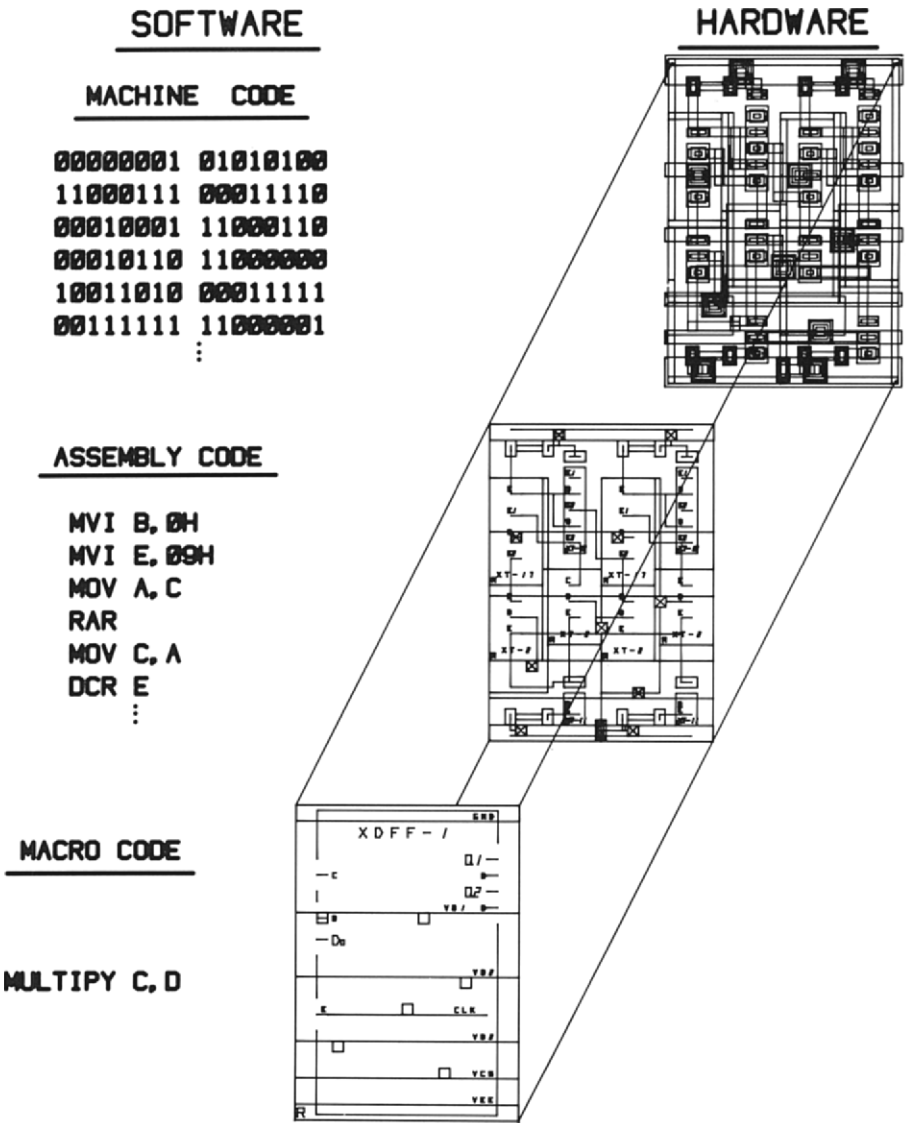


Fig. 3. Illustration of the software–hardware development analogy.

(4) The mapping from a high-level symbolic graphic description to the physical mask description is accomplished by a chip assembler and is left until the end of the design process.

An important part of the ASAP design system is the mapping between the physical description and a higher-level (symbolic) description. This mapping requires close cooperation between chip designers and process development teams. This gives a unique advantage to Hewlett-Packard with its captive IC shops. This cooperation is a departure from the normal situation in which the process development team generates the physical design rules for a particular IC process, based on the device physics and other factors (e.g., photolithographic capability). In the past, little consideration has been given to how a chip designer might implement the IC design. While adhering to process-based design rules may yield the smallest chip for special circuits, such as memories, it is more important for the design rules to be self-consistent in a structured design methodology based on reduced low-level data and to allow increased emphasis on global optimization.

The IC designer and the process designer should interact during process development, because the process designer's objectives should be compatible with the chip-design methodology. As will be shown later, the smallest layout often results from optimizing the overall design concept rather than from minimizing design rules. The application of symbolic mapping offers great returns by improving the designer's global perspective without losing local cell resolution.

What has the new mapping accomplished? It has generated a ''gridded sticks'' graphics representation describing the physical aspects of the design that can be bidirectionally mapping between the physical plane and the symbolic plane. The working scale has been reduced, and superfluous physical information has been eliminated.

Capturing the physical, structural, and behavioral representations of an IC is an important step toward correct design. An IC design specification normally describes a chip's structural and behavioral properties, and most design systems force the IC designer to translate these descriptions manually into physical layouts. Unfortunately, these physical layouts do not contain adequate information to allow automatic design verification, which generally requires that the structural, physical, and behavioral attributes of the design be known to the design system. The ASAP system includes these attributes in its hierarchical data base.

Because ASAP designs are built with a hierarchy of components, a component may be as small as a single via or transistor or as large as a collection of such primitives. The structural (and some behavioral) aspects of a component are captured along with the physical description. The external structure and behavior are defined by the component's physical boundaries and interconnection points. Each component is named, and the interconnection points are defined. The definitions appear in the graphic descriptions for visual reference and in the data base for use in design verification. The component boundaries and interconnect points must lie on the grid, although any design rules can be used to describe the "guts" of the primitive components.

In addition to the primitive components, there are high-level component symbols. These symbols are *prime,* if the component symbols are available from the system library and are of predetermined geometry, or *parametric,* if the component symbol often changes size in the layout and if an algorithm for mapping that particular parametric component exists in the assembler. All parametric components have unique names and are recognized visually by their unique geometry. An algorithm in the assembler contains their structural and behavioral descriptions. In order to use an instance of a component symbol correctly, a boundary must be defined for each mask level. These boundaries are an abstraction of the internal geometry of the component and indicate legal wiring areas. Symbolic circuit design begins after a set of the most primitive symbolic components has been established for a technology.

Figure 4 contains a flow diagram showing how components are combined with symbolic interconnections to define a symbolic layout. User-created macrocells are blocks defined as being local to a specific designer's file. Prime components are created to update the system library and to make new prime components available to all users. The conversion from symbolic layout to final artwork by the assembler occurs after design, editing, and verification are complete.

Observe the three different hardware descriptions of a bipolar D flip-flop in Fig. 3. The middle description is a symbolic layout using primitive components defined for the process. This layout was designed following the flow diagram of Fig. 4 and contains all the information necessary to complete the design and to verify the flip-flop. Verification includes topology (or continuity) checks, process design-rule checks, and network extraction for input to a logic or circuit simulator. If the assembler processed the layout at this point in the design, the physical mask description shown at the top of Fig. 3 would result.

After the module is designed and verified, a higher-level description can be defined, as shown at the bottom of Fig. 3. This graphic description contains all the physical mask boundaries needed to instance the flip-flop into a larger network. The structural and behavorial descriptions are raised to a higher level, enabling the assembler to link the previously generated flip-flop mask description to the higher-level description. All the rules for defining primitive components are used to define this new, higher-level component symbol. The resulting macro- or prime symbol can be instanced and connected using the same interconnection design rules that govern primitive components. Thus, the designer can meet the original goal of reducing information while retaining all other useful attributes of

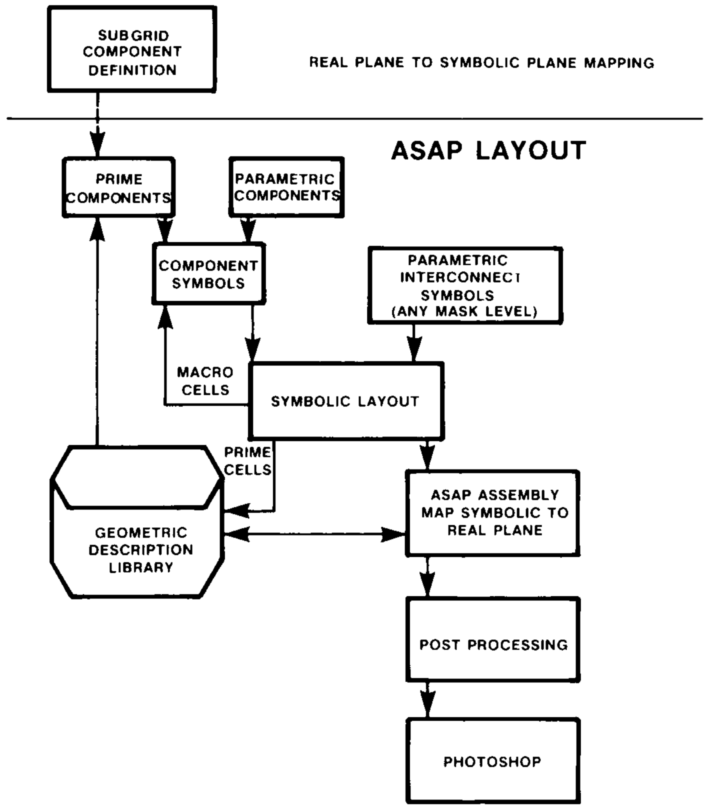


Fig. 4. Flow diagram of symbolic layout process.

the methodology. The time required for chip design and composition can be minimized by using components at the highest possible level. It is important to understand, however, that ASAP allows consistent definitions and simultaneous utilization of high-level and primitive components.

A structured design approach has been used to create a library of regular block structures. The blocks have a matching cell pitch and bus structure in the vertical dimension and vary in width to accommodate a complete family of structured cells, as illustrated by the D flip-flop in Fig. 3. Instances of similar cells with well-defined interfaces are placed in a regular pattern to form larger blocks. The final block represents the complete design description. A critical difference between these structured cells and "standard" cells used by others is that the structured cells are designed with the architecture and interconnection needs of the system in mind. Thus, most of the wiring management is handled over the cells before they are placed rather than afterward.

B. The ASAP Design System

The ASAP design system and tools have been developed to address the complete design cycle. The three subsystems within ASAP are shown in Fig. 5.

The IC device and cell definitions are critically important to the design cycle. A hardware measurement set allows ac and dc process and device

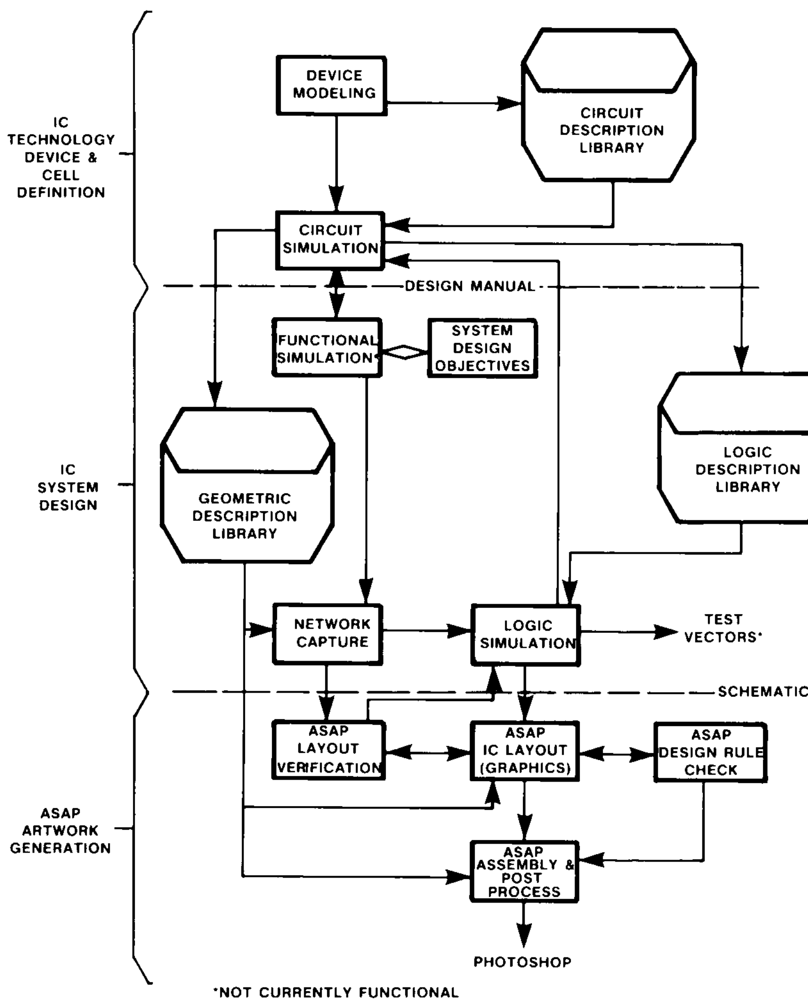


Fig. 5. The ASAP design cycle. The asterisk (*) denotes those operations not currently functional.

parameters to be determined. The parameters are used to generate models for a circuit simulator (SPICE). A library of structured cells has been designed, and descriptions have been placed in the geometric description and logic libraries. The generation of symbolic designs for the library follows the design process shown in Fig. 4. The descriptions contain structural and behavioral data, all cells are named, and the entire interface structure is defined by pin names and boundaries. In addition to identifying the cell's interface pins, the names use a convention to include type information. In this way, behavorial information is added to the interfaces for checks, for example, for consistency in logic-family design rules. These libraries are documented in a manual used by the system designer, who can contribute cells to the library or create user-defined macrocells.

The architecture of a particular digital system is designed during the IC system design phase. A functional simulator would allow the designer to define the system design objectives in terms of a high-level structural and behavorial specification. The schematic is entered into the network capture system by means of an interactive graphics system from which net lists are generated. The net description can be input into the logic simulator, where it is combined with the rest of the behavioral description to check the performance of the design. If the desired performance is not achieved, new cells may be generated. With the aid of ASAP, system designers without detailed knowledge of integrated-device physics can specify designs successfully.

The completed schematic design is entered into the artwork generation cycle. Chip composition is accomplished as previously defined in Fig. 4. The designer uses a conventional graphics system to manipulate the symbolic structures and complete the symbolic design. The design-rule check (DRC) is used on the symbolic layout to find and correct violations in the physical layout. Because the composition is verified in each design step, the information to be verified is limited to that added in each step. After the chip is composed, its node list is compared to the previously entered schematic node list to reveal layout errors. All discrepancies are listed, and the layout is corrected. The dashed line into the logic simulator shows a verification path for information extracted from the symbolic mask description along with the interconnection parasitics. Combining the network and parasitics with the behavioral descriptions from the logic library provides a final design check. The corrected design is assembled into final detailed artwork and is postprocessed for input into the photoshop. The CPU time on the minicomputer-based graphics system for chip assembly and topology verification is typically 5–10 min.

The ASAP system has been used to complete approximately 20 custom chip designs, supporting a wide range of products and applications in

TABLE I

Comparison of ASAP and non-ASAP Circuit Designs

	Non-ASAP					ASAP				
Chip	1	2	3	4	5	6	7	8	9	10
Type[a]	REG	RAN	REG	RAN	REG	RAN	REG	RAN	REG	RAN
Layout method[b]	REC	REC	REC	REC	REC	SYM	SYM	SYM	SYM	SYM
Chip area (mm^2)	9.7	12.3	10.2	9.0	14.8	9.6	8.7	13.7	13.2	13.5
Layout time	40 weeks	40 weeks	15 months	32 weeks	15 months	22 days	17 days	25 days	22 days	30 days
Components/day	8	8	5	9	9	80	103	103	138	76
Component density (comp/mm^2)	165	125	180	155	200	195	210	200	230	170

[a] REG = regular architecture; RAN = random architecture.

[b] REC = rectangular; SYM = symbolic.

analog scopes and displays, A/D convertors, and digital circuits for logic analyzers. Table I compares the results of a cross section of five ASAP designs to those of five previously designed circuits. Both sets of circuits use the same logic family and address similar design needs. Each group contains a cross section of random and regular architectures. Average chip composition time has dropped from 46 weeks to 24 days and the average number of components designed per day has increased from 7.8 to 100!

The structuring of the design process does not have to reduce component density. As previously stated, global perspective and wiring management (the most important aspects of complex designs) have been improved by the reduction of all graphic information not related to interconnection. The ASAP chip design emphasizes wiring management. The power bus and wiring strategy of the structured cells establish a common but efficient strategy for the entire chip.

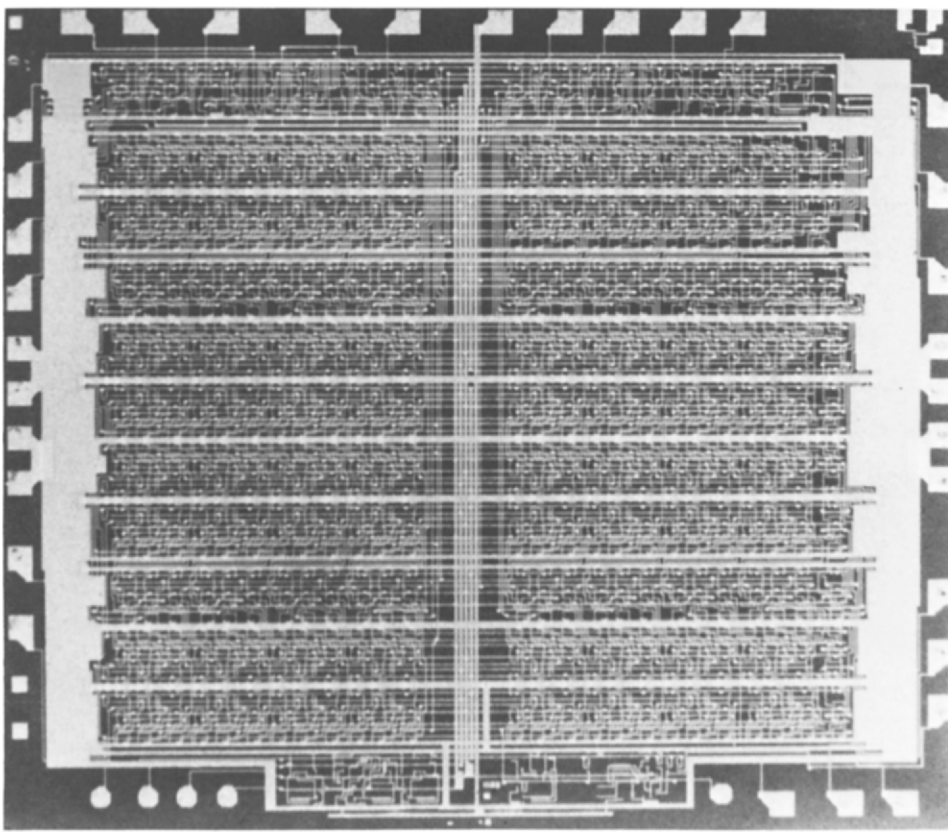


Fig. 6. Chip designed with ASAP-structured cells.

The last row in Table I points out a very interesting characteristic of the ten circuits. The component density has in fact increased over that of handcrafted circuits. Figures 6 and 7 show examples of fabricated circuits designed with ASAP. Figure 6 shows the structured power distribution and wiring of a circuit design using structured cells, and Fig. 7 shows a circuit with a more regular architecture, which was designed by combining structured cells at the chip periphery with a lower-level description of the center array area.

C. A System Design Example

The next generation of the Logic Systems Operation product line for logic analyzers will be used as a design example of the use of a workhorse technology and the ASAP design methodology. As indicated previously, this generation of equipment was postulated to need custom integration to

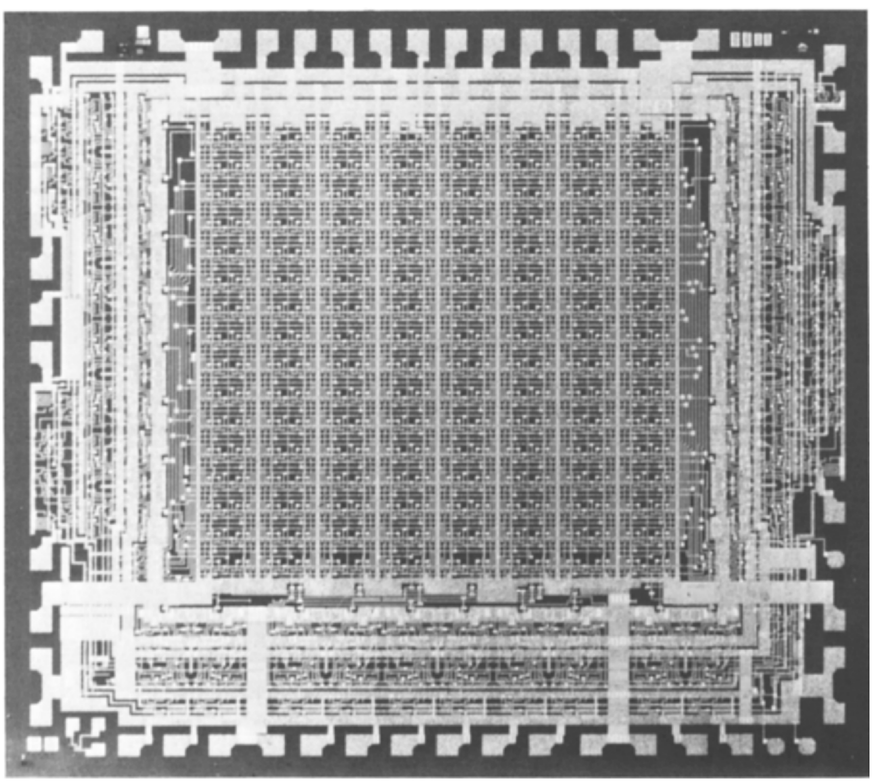


Fig. 7. Chip designed with ASAP-structured cells and low-level primitives.

be feasible. Also the necessity of allowing much more permutation capability for customer selection of feature sets meant that the overall system design would be an enormous task. This would require the design team to be looking at the forest and not at the trees. Custom ICs had to be used because of the power, speed, and complexity requirements of the systems to be described, which are subsets of a larger digital systems "design center." The ASAP methodology allows the systems designers to spend most of their time in the creative thought process of system feature set and not to be dominated by the "tooling" of the IC mask sets. A brief overview of the logic timing analyzer and the logic state analyzer will be given. A custom IC for each design will be discussed in more detail also.

The block diagram of the timing analyzer is shown in Fig. 8 and is rea-

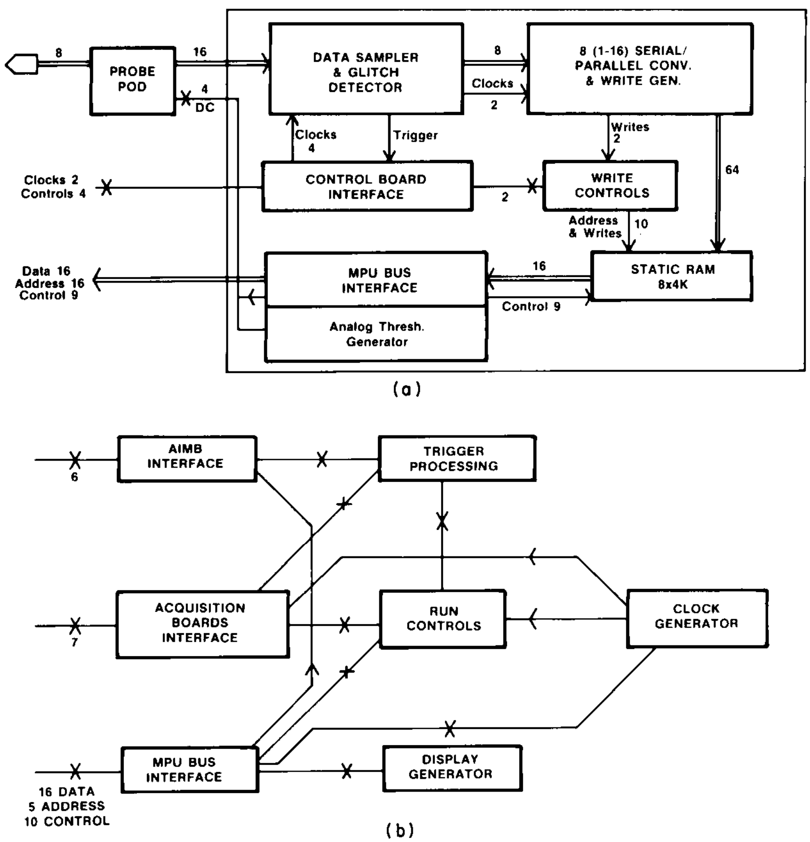


Fig. 8. Block diagram of logic timing analyzer: (a) 200-MHz data acquisition board; (b) timing control board.

lized with custom ICs. Eight channels of the timing analyzer are realized on 2 PC boards with 14 custom IC's. The logic timing analyzer samples input data present at its input with a periodic, internally developed sample clock, i.e., the sampling is done asynchronously from the system under test. High sampling rates are used to give the user the best time resolution possible.

The inputs to the timing analyzer are compensated probes, which drive an 8-channel custom comparator chip. Having all 8 channels on the same chip allows close delay matching without adjustments. The outputs of the comparators drive a high-speed emitter function logic (EFL) custom IC which samples all 8 channels, again on a single chip for low timing skew and excellent high-frequency performance. The highest sampling rate is 400 MHz. The high-speed sampled data are input to a custom serial-to-parallel convertor, the *encoder,* which slows the data rate down to 12.5 MHz maximum. The 12.5-MHz outputs drive commercially available static RAMs for data storage.

The custom chip set for this instrument allows several acquisition modes: normal, dual threshold, glitch capture, and complex asynchronous trigger capability.

The encoder is a series-to-parallel converter for use with the timing analyzer data sampler IC. It effectively reduces the data rate to allow the storage of timing data in conventional fast RAMs. It has two basic modes of operation plus a test function.

The first mode of operation is the conversion of a single ECL series input to 16 TTL tristate parallel outputs. The outputs are split into two groups of eight, each with an associated write-enable pulse output for controlling RAMs. The data on the output lines is changed on the rising edge of the WRT-not pulse. At the maximum input sample rate of 200 MHz (5 nsec), each WRT-not pulse has an 80-nsec period (12.5 MHz), putting it in the range of conventional fast bipolar or HMOS RAMs. The input sampler for this mode works on both edges of the sample clock, which is less than or equal to 100 MHz. In the serial mode, the sampled data is stored alternately in two, two-phase, 8-bit shift registers before being latched into the outputs.

The second mode of operation converts the serial data from 4 ECL parallel inputs to 16 TTL tristate outputs, split as before. In this mode, the maximum sample rate is 50 MHz (20 nsec)/channel to produce an 80-nsec write cycle as before, but the input sampling is only on the positive sample clock edge, making it less than or equal to 50 MHz. In the parallel mode, the sample data is stored in the same shift register elements. In effect the registers are reconfigured into four 4-bit registers.

In the test mode, an alternating checkerboard pattern is output statically on all 16 TTL outputs, and the WRT-not outputs are low. This permits a partial self-test of the IC plus loading of a RAM test pattern.

Figure 9 shows a PC board with an off-the-shelf ECL realization of the encoder design. It was used as a pin-compatible emulation of the encoder during the feasibility stage of the design. In front of the PC board is the custom IC encoder. Table II illustrates the advantages of custom IC design and the ASAP methodology. This example is typical in terms of the advantages for all the custom ICs designed in the Logic Systems Operation. Eight encoders are used per 8 channels of timing analyzer. The power savings and PC board area savings are significant when coupled with a much higher performance and reliability.

The state analyzer is used to monitor information flow in the data domain. This information may be a software program, the actions of a hard-

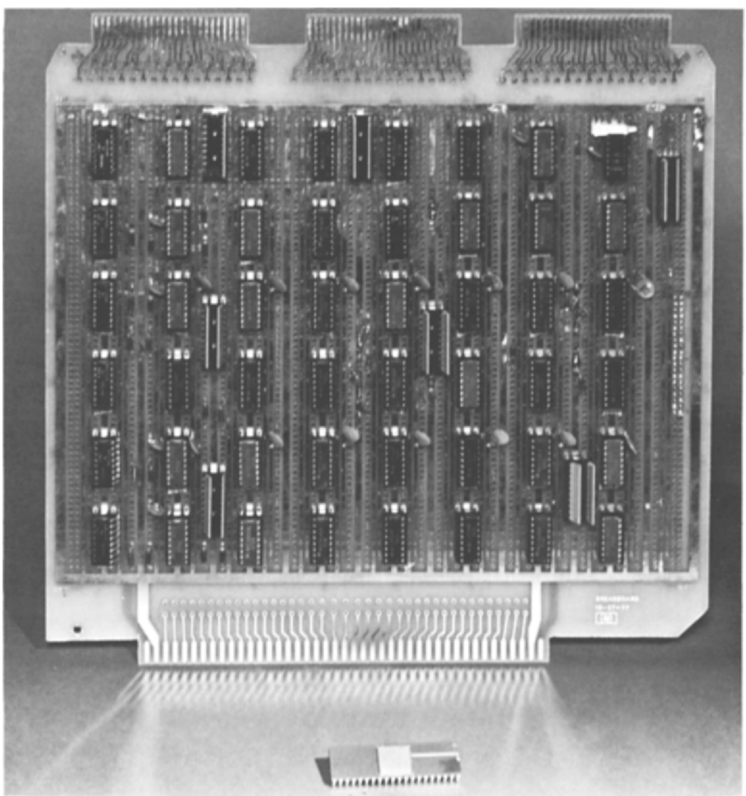


Fig. 9. Encoder discrete and custom IC realization.

TABLE II
Encoder Discrete and Custom IC Comparison

Parameter	Multilayer PC with off-the-shelf ECL	Conventional layout with custom LSI	ASAP/structured cell with custom LSI
Number of dips	48	1	1
Power (W)	10	1	1
Turnaround time (weeks) (to first parts)	12	40	16
Required PC board area (in.^{2})	80	2	2
Performance (MHz) (sample rate)	40	200	200

ware state machine, or just random logic signals. In any case, the ability to monitor the flow or sequence of states allows verification of good activity as well as the detection of improperly functioning hardware or software.

The block diagram in Fig. 10 shows how a basic state analyzer works. The logical information on probed user signals is transferred into an input data register by a data strobe derived from a user clock. This user data are then available for storage in the data memory. Events, which are 1, 0, X patterns or sequences of these patterns, are also detected in the user data information flow. These events are then used to control the basic functions of the state analyzer—trigger, storage control, and count control.

The *trigger* is the point in the state flow at which it is desired to see the sequence of states. This point may begin storage or end storage in the memory. In either case, it is the reference point for viewing information.

Storage control is the process of filtering the incoming states and allowing only those desired for viewing to be stored in the memory. When a state is to be viewed, it is called a *store-qualified* state and is written and saved in the memory.

Along with each state stored in the data memory, an associated *count* is stored in the count memory. The stored count may represent either the time between stored states or the number of states between stored states. The display presented to the user can then include, not only the flow of states, but also the time or number of states that occurred between each state.

In the 10-MHz state analyzer, custom ICs are used to perform three

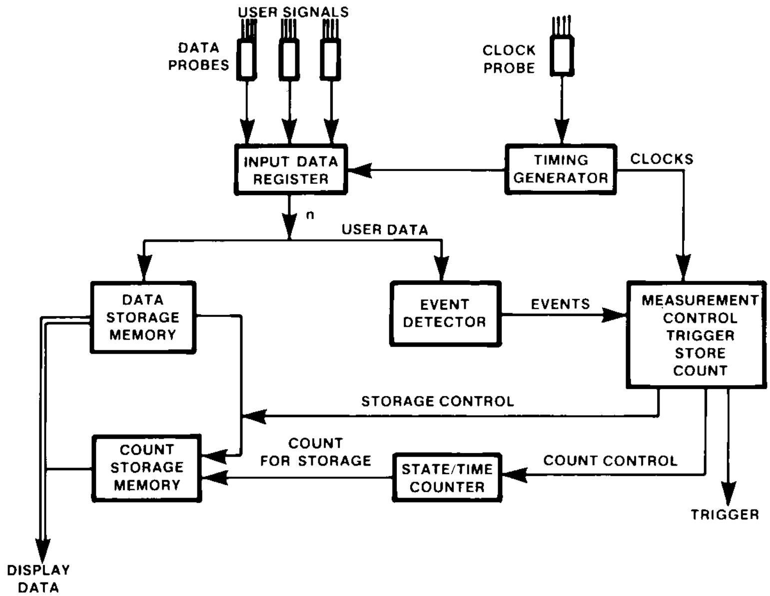


Fig. 10. Block diagram of logic state analyzer.

functions. They are

(1) clock term generation in the timing generator,
(2) the state/time counter, and
(3) many of the measurement control functions.

The state/time counter used in the logic state analyzer is a floating-point gray code counter. The counter provides a 20-bit output with a 3-bit exponent and a 17-bit mantissa. It will operate either as a time counter or as a state counter. The time count mode provides a minimum resolution of 40 nsec with a minimum 3-digit accuracy from 100 nsec to 30 ksec (8.34 hr). The state count mode provides single-state resolution to 612K states with reduced resolution out to 750 gigastates.

The floating point counter was designed to be used in two applications by charging the pinout connections.

1. Micro trace time/state count[1]

(a) State count. Count-qualified states between each storage-qualified event. The state count resets on each storage event and is thus relative, state to state. Due to propagation delays, the state count is accu-

[1] This refers to the standard trace measurement where store qualified states and related information are stored relative to a trace point.

TABLE III

Custom IC Realization Comparison

Feature	New state analyzer 64620A	Model 1610A
Probe width	120 Channels max 256 Deep trace memory	32 Channels 64 Deep trace memory
Clock options	Logical combination of 8 clock channels qualify and enable on level or edge	Logical combination of these clock channels Clock channels "ored" Demultiplexing
Sequence terms	3 Modes: 16 deep sequence, 1 window + 8 deep sequence, 2 windows + 4 deep sequence	7 Deep sequence
Trigger on	Stimulus from another analysis module Sequence control 8 "Ored" patterns 4 Ranges 4 Exclusive events Overview event	None
Store on	8 "Ored" patterns 4 Ranges 4 Exclusive ranges Externally applied window Sequence window Sequence terms stored by option	None
Count on	4 Patterns 2 Exclusive events 2 Ranges time event (All windowed by sequence or external analyzer.)	None
Variable names	User-definable alphanumeric strings such as: ADDRESS, DATA, CARD Alphanumeric strings such as: START = 08002H, INTRP = 0000H, variables or constants	6 Predefined names: A,B,C,D,E,F
Overview analysis	Tracing up to 15 events or ranges to generate a trace list or histogram Histogram of execution time distribution from start to stop Trigger on illegal event or unexpected time of execution	None

rate only through 17.3 gigastates. Beyond this point, the count is uncertain to ±1 prescale clock, or 16.7 megaevents.

(b) Time count. Count 25-MHz clock. The count is reset when the stored count is beyond the midpoint of the divide by one region of the counter. The time count is therefore absolute for store-qualified states occurring rapidly and relative for states with greater separation in time. The time is measured from write strobe to pipeline strobe. Thus, counter time = clock time − (67 ± 23) nsec.

2. Overview count resource.[2] Similar count function to microlevel except count is begun by start event from sequencer and ended by stop event also from sequencer. The counter outputs are applied to the range hardware.

(a) State count. The state causing reset can also be counted after the reset. The state before the stop is not always counted. (Whether or not it is counted is a function of incoming data rate and the position in the count.) The state of the stop event is never counted. Due to propagation delays, the state count is accurate only through 17.3 gigastates. Beyond this point, the count is uncertain to ±1 prescale clock, or 16.7 megaevents.

(b) Time count. Time is counted from overview reset to state recognition. Thus counter time = clock time − (145 ± 25) nsec.

The state/time custom IC provides advantages in reduced power, reduced PC board area, and increased performance as did the encoder custom IC. Also, the user feature set is enhanced as shown in Table III with a comparison of features in the new state analyzer (HP64620A) and the 1610A. Figure 11 shows the hardware realization of the logic state analyzer on top of the new generation mainframe into which the two PC cards fit.

D. ASAP Versus Other Methods

With the complexity of ICs increasing and the design times becoming longer, various methods are gaining acceptance for addressing the long design times. Most of the methods give up functional density to achieve a quicker design time. The fact that much higher complexities are available today has made these methods feasible, especially for low-volume applications. Table IV compares some of these methods relative to the ASAP approach. The problem in the past has been the relatively low complexity

[2] Measurements are made on state data without storing the raw data but rather storing condensed data.

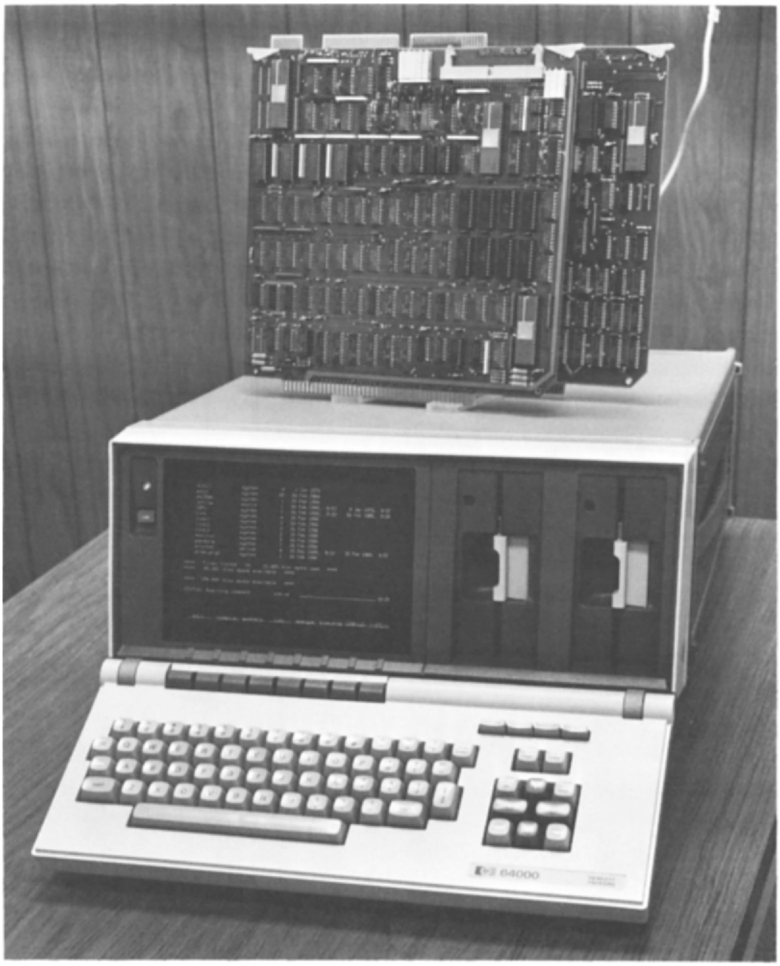


Fig. 11. Logic state analyzer (64620A) PC cards on new-generation mainframe.

capability of gate arrays and PLA/PSMs. It was difficult to partition a system into the complexity limitations. As future processes provide higher complexities, it will be less of a problem and each method will have a unique place in custom IC design. The significance of these methods will be determined by the *design aids* developed to allow the utilization of VLSI's inherent complexity and power.

Some comparison of the complexity of this IC family may be realized by comparing it to other HP proprietary IC families. Figure 12 shows the number of chips and ceramic hybrids that exist in the family today, and

TABLE IV

ASAP Comparison to Other Custom Methods[a]

Feature	ASAP	Commercial gate array	Commercial PLA or PSM[b]	Conventional full custom
Functional density	1	0.5×	0.25×	1×
I/O Flexibility	Full	Limited	More limited	Full
Design time (to first chips)	1	0.75×[c]	0.25×	3×
Speed (gate delay)	1	>1×	≫1×	1×
Power/gate	1	>1×	≥1×	1×

[a] In this table, a common process technology for all methods is assumed.

[b] Assumes mask programmable, not field or fusible-link programmable.

[c] IBM has reported much faster turnaround for their proprietary gate array capability.

Table V compares the family to other previous HP custom chip sets. It is important to appreciate that this family was considered workhorse in 1979–1981, while each of the comparison groups were considered state of the art at the time they were developed. Thus, it should be clear that we are really beginning to be able to design high-density, high-performance, low-volume custom chip sets in comparison with very recent performances that had much higher development costs and involved more designer difficulty.

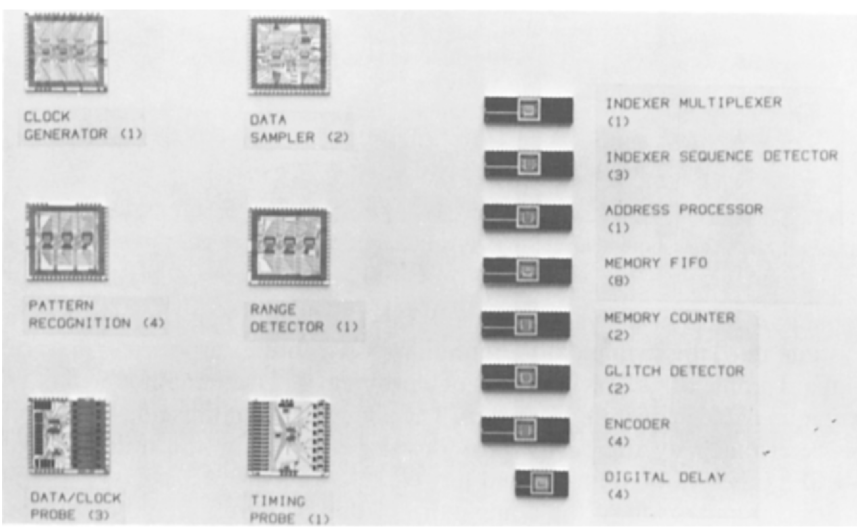


Fig. 12. Logic systems' custom IC chip family second-generation use in the Gemini project, totaling 36,000 devices and 15 chips.

TABLE V
Comparable HP Custom Silicon Designs

Design	Process	Equivalent gates	Throughput (clock × equivalent gates × 10^{-10})
5315 A/B Universal counter	SCLSI	1600	16
3455 Voltmeter	NMOS 1½	3300	3
3585 Spectrum analyzer	NMOS 1½	8300	9
HP 300,3000/33 16-Bit processor	SOS 1	8300	10
9825/45 16-Bit processor	NMOS 2	15,000	15
Logic systems, logic analyzers	LSI 1	12,000	42

We have referred at length to the ASAP process of structured cells to be interconnected in a gridlike level. This obviously is a graphics approach, totally at variance with the automatic artwork generation systems being developed today for both IC and PC layout systems. It is also clearly an intermediate solution, appropriate for workhorse and today's complexity but not state of the art where capital equipments can be lavishly spent. This brings us to the last point of major consideration, as it is being experienced at Hewlett-Packard.

Design aids are clearly being developed at a rapid pace, but they are also being developed in somewhat dichotomous directions. We mentioned previously the alternative approaches to IC layout design centers, with our graphics solution being a "less preferred" choice by most who deal with state-of-the-art concerns. This differentiation continues into software development, simulation, analysis, factory test, and field support as well as IC and PC design.

The distinction again revolves around economies of scale. When volumes are sufficient to justify robotics and massive automation of centralized design processes for singularly important chips or assemblies or test problems, there is compelling reason to invest in the most automated of equipment, which precludes graphical solutions, iterative manual test methods, or even iterative professional design time. Wherever workhorse silicon design is appropriate for the reasons that we have previously cited, it seems to correlate within our company that the design teams are continually changing their focus, their tasks, and their tools. The premium is on flexibility and versatility rather than on the ultimate performance, just as it is in the silicon process as well.

This has led to the discovery that the computation revolution has moved far enough already that "reasonably" performing comprehensive design centers may be created that link the work of professionals across many functional boundaries. This provides many advantages of improved design tools, but much more importantly for design productivity, the work is correlated, documented, and traceable throughout the entire project. The trade-off for the latter is so vital that some significant reduction in the specific quality of the individual design aid has been acceptable to date.

We might illustrate this with an example from software development. In Brooks' classic book "The Mythical Man-Month" [3], it is carefully noted that communication is the bugaboo of design effectiveness. The computer companies have known this for years and have installed linked, time-shared, computer development centers in order to facilitate their tasks. The microcomputer systems people, by contrast, were typically hardware designers, more concerned about the cycle times, interrupt rates, and other electronic parameters than they were about data rate and control. Significantly, as the first generation of microcomputer development systems were created by the semiconductor manufacturers to help their clients over the "software hurdle," the products focused heavily on the "emulators" of the hardware chip set, the better to evaluate race conditions, marginal utilization, and feature set infringements. They paid no heed to the problems of significant code space, of several designers jointly developing codes, or of documentation and back-up.

Within Hewlett-Packard, the move to algorithmic state machine design was well along, due to the examples of the original scientific calculator design techniques and Clare's excellent book [4]. Moreover, the computer divisions had considerable software experience, and when the company sought to enter the microcomputer development system business, it did so by pioneering a second generation of equipment, putting the emphasis much more heavily on multiuser, multitask, shared-data file management systems and to a large degree deemphasizing the hardware aspects of emulation in favor of a more comprehensive and balanced approach to the time utilization of the professionals faced with the total design task.

Thus, in effect, in software development system tools, there has evolved an intermediate solution, inferior to the large systems extant at the major computer companies for huge software tasks and only matching in some ways the excellent pulse stimulus and tracing functions of the stand-alone testers. Importantly, it is superior to both for the class of user that it addresses, to wit, the design team with significant unique user application code to generate, with relatively "clean" non-state-of-the-art hardware techniques. Of course, this parallels the workhorse silicon

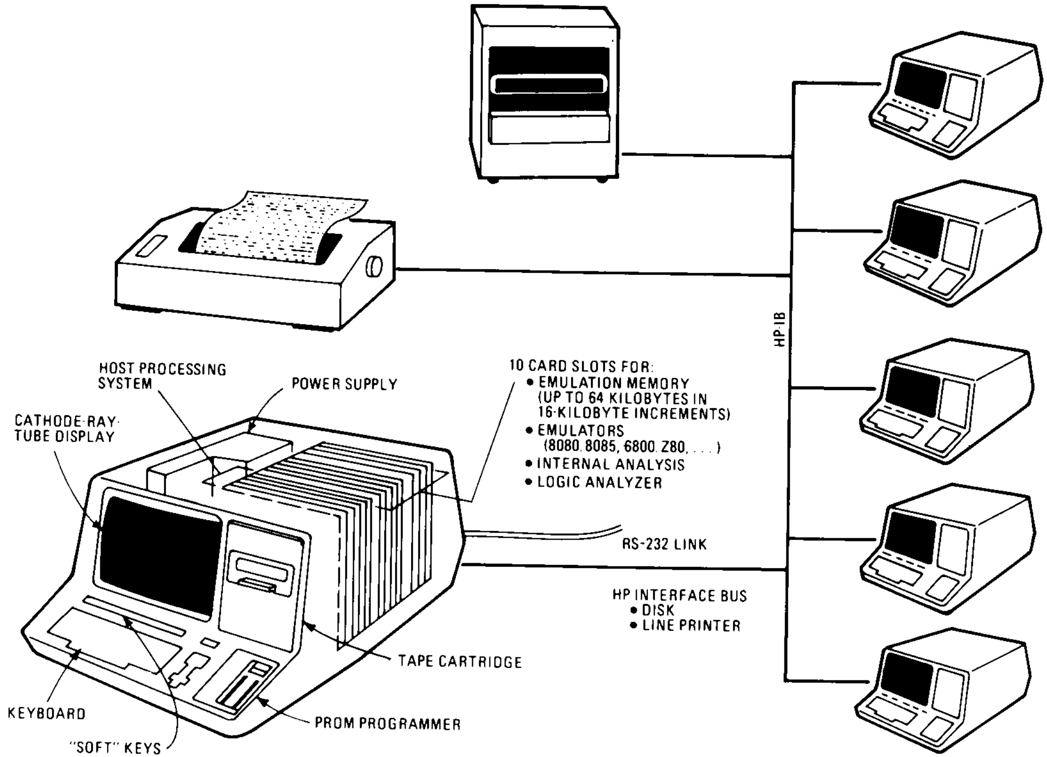


Fig. 13. Design-aid center cluster—HP64000A.

examples delineated throughout this chapter. The solution is based on a cluster of work stations, as shown in Fig. 13.

VI. THE FUTURE

Where do these thoughts lead us? What will the design aids of the future look like for such environments? First, of course, it is worth citing our belief that Mead and Conway [5] are correct in saying that "the ways in which digital systems are structured, the procedures used to design them, the trade-offs between hardware and software, and the design of computational algorithms will all be greatly affected by the coming changes in integrated electronics" (p. 5). Given this, we would observe that the "electronic bench" is fast becoming a reality in many companies, albeit from ill-fitting and inefficient parts today but, nonetheless, filling a great need.

What is the electronic bench? Basically, it is a design aid that is resident on the designer's desk or bench that is essentially always recomfigurable for the design task at hand. Consequently, it must be able to handle synthesis tasks (software development, IC layouts), analysis tasks (tracing, test patterns), documentation, and linking to other members of the team. The linking is vital in order that both information be shareable and the total resource capabilities be shared. To illustrate this concept, Fig. 14 portrays a typical product life cycle for a digital product, along with the classical design aids and analysis tools used by most companies today. There are several points worth noting:

(1) Virtually all design aids and analysis tools in use today are not linked in any data-base or even measurement-interactive manner.

(2) The level of synthesis capability in the design aids is extremely primitive.

(3) The level of zoom from "high-level" to "low-level" analysis is likewise primitive.

(4) The operator interface is quite variable and quite formidable from one piece of equipment to another. If we examine the needs that VLSI design imposes, this is clearly unacceptable.

There are some current examples of the electronic bench concept at such places as automotive design research centers, airframe manufacturers, and the larger computer and semiconductor design centers. These centers, usually focused around computer-aided drafting systems, are very expensive but also very productive. Just as the computer mainframe and minicomputer manufacturers developed precursor equipments of the

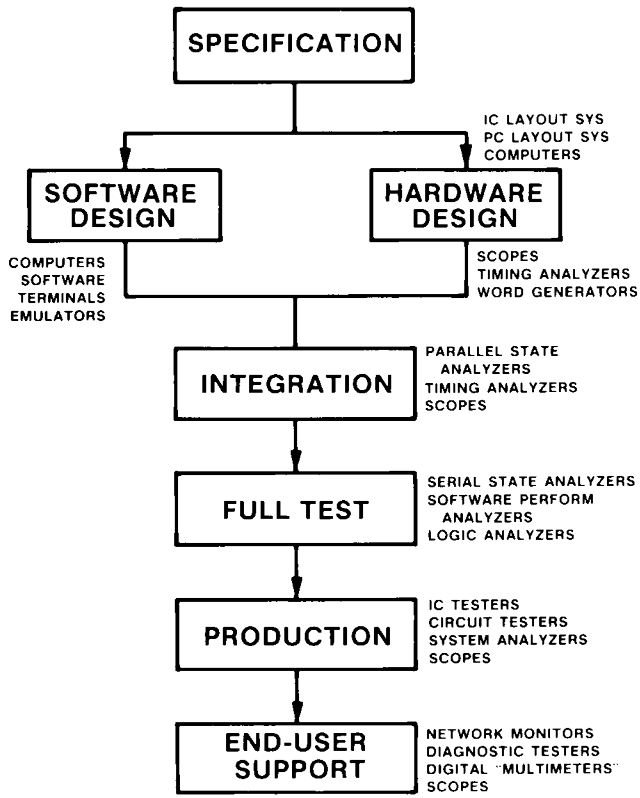


Fig. 14. Product life cycle—digital systems.

linked-data type of software development system, these CAA and CAD centers point the way toward the electronic bench [6].

In effect, the solution will embody an intelligent terminal or work station that can provide the capabilities of any required design aid or analysis function. Work performed at this work station will automatically link to a shared data base for the entire program, which includes the R&D functions, production tests, service diagnostics, and documentation. Likewise, environmental and life-test data will become the beginning library of service data that links with lab analysis, production data, and user performance data to promote design improvements and better field-support diagnostic procedures.

It is not hard to postulate such capabilities or their desirability. What has been difficult is a cost-effective and performance-effective realization [7]. There are three major handicaps in this regard when we examine the realities of existing digital analysis tools, to say nothing of the shortage of effective synthesis tools.

(1) The user interface of most instruments is very complex and the commonality of terms, function, and operation is very low. For example, the specific functions available by name on the front panel of a storage oscilloscope, a logic timing analyzer, and a serial data bus analyzer bear little resemblence to each other. Each takes considerable "getting used to" for a beginning operator, and knowing one of them well can often seem more a handicap than a help when trying the next machine.

(2) Today's realizations of this equipment are sophisticated, reasonably expensive, and relatively bulky. The thought of creating an "integrated" solution has historically been dismissed, from a practical standpoint, in terms of size, heat, weight, and cost.

(3) Linking many measurement hierarchies in the zoom context has not been required or practical, because of the preceding facts about the available instrumentation and also because the problems being tackled were able to be solved by brute-force techniques.

The architectural concept of the linked software development system shows a potential solution to some of these points. The foremost problem, the human interface, is addressed via a standard typewriter keyboard, along with a "guided syntax" and "soft key" format. The versatility of screen graphics for menu selections or guided prompting is well established in instrumentation by now; it is a simple extension to provide conversion from one basic type of equipment to another. The difficulty with such a concept is the reality of its implementation.

Let us consider the manner in which the guided syntax structure operates. The guided syntax soft keys represent another important enhancement of the soft key with "help" augmentation embodied in many of the industry's more recent computer systems. Not only do these keys provide "prompting" of the next correct or allowable entries, along with recursive parsing for erroneous entries, but they allow full flexibility for system reconfiguration as the resident "operating system" module is swapped from the disk.

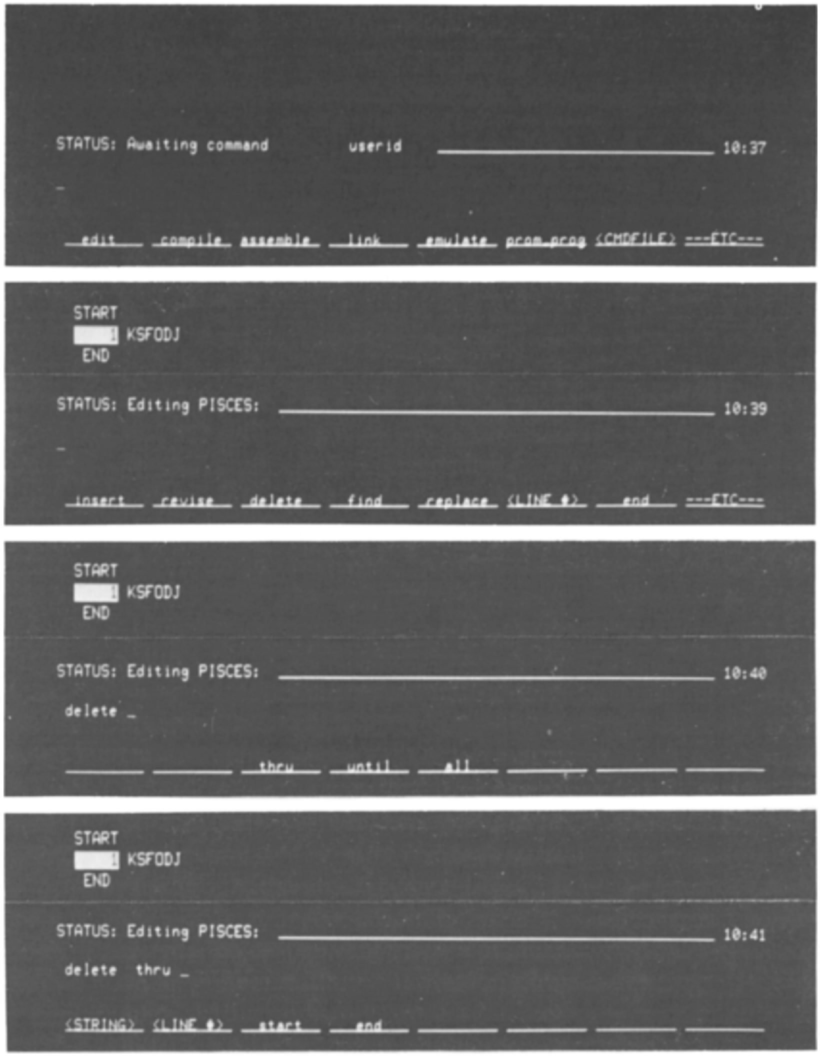
For each work station of the cluster shown in Fig. 13, there are eight soft keys underneath the CRT (Fig. 15), which are labeled dynamically on the CRT itself just above the keycap. The operation of the system might be as follows. After logging onto the system, the station will be in a *Monitor* state. The Monitor provides three levels of soft key scrollings for an extended menu of functional modes from which the user may select. For example, some basic choices available from the Monitor include Edit, Compile, Link, Emulate, PROM Program, and Command File. Assuming

Fig. 15. The HP64000A software development system.

that we would like to edit a previously composed program, we would press the Edit key, whereupon the soft key labels would all modify to allow the appropriate edit functions to be selected by the user.

The Edit program is overlaid into the station's RAM from the disk in place of the Monitor program when the key is pressed. This use of the disk for the extended features of the operating system software is but one of the advantages available through the use of the large disk and the high-speed data bus. Edit choices now include Insert, Revise, Delete, Find, Replace, Merge, Copy, Extract, Retrieve, Renumber, Repeat, Tabset, and List among others. If we press Delete, for example, it offers several choices as well, including Thru, Until, and All. If we press Thru, the labels then allow a ⟨String⟩, a ⟨Line #⟩, Start, or End (Fig. 16). Other key choices are equally specific in prompting the user through sets of commands that are syntactically correct for difficult sequences as well as easy ones. The significance of the syntax-driven keys is hard to overestimate.

Notice the significance of this architecture. The stored program that



(a)

(b)

(c)

(d)

Fig. 16. Key feature. Below each of the underlined words on the 64000's CRT screen is a key whose function is flexible, determined by software. Thus a user is easily guided through the editing process, shown in successive photographs, by changing the words to indicate choices.

determines the machine characteristics appearing to the operator is totally resident on a disk. Thus, *redefining the instrument is easy,* and the operating system's reconfigurattion time is about one-third of a second! Moreover, the guided syntax appreoach removes the need for a different set of keycaps on the front panel, and the user never is faced with relearning the panel functions as the instrument changes.

Thus, such a work station cluster architecture links all data files, provides redefinition of effective functions at each work station, and allows easy operator interaction with those significant changes. The major remaining tasks are twofold: (1) to provide extended operating system enhancements in the guided syntax format and (2) to provide data acquisition modules for specific functions that may be required. This flexibility could be employed as a local computer resource or as a distributed system emulating terminal for any computer system, as the following whimsical soft key choices illustrate:

Local computer | Remote terminal emulation

When remote terminal emulation is pressed, the choices are

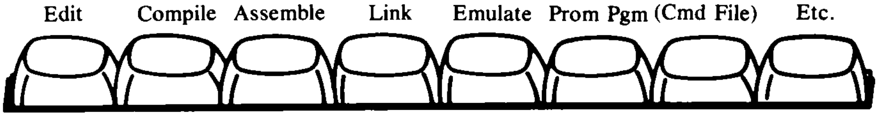
HP 9845C Graphics | HP 1000 term | HP 3000 term | IBM term | DEC term | APPLE term | Personal computer | Etc.

When local computer is pressed, the choices are

Software develop | IC | PC | Digital analyzer | Analog analyzer

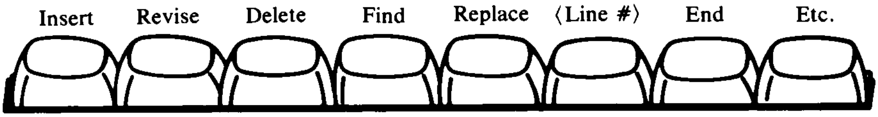
When software develop is pressed, the choices are

Edit | Compile | Assemble | Link | Emulate | Prom Pgm | (Cmd File) | Etc.

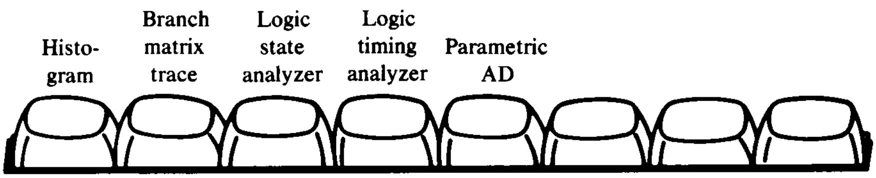


When Edit is pressed, 32K bytes of operating system code is swapped, and these become the key labels:

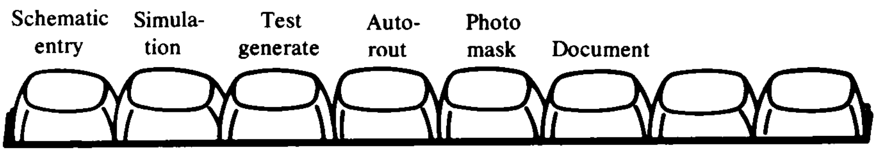
Insert | Revise | Delete | Find | Replace | ⟨Line #⟩ | End | Etc.



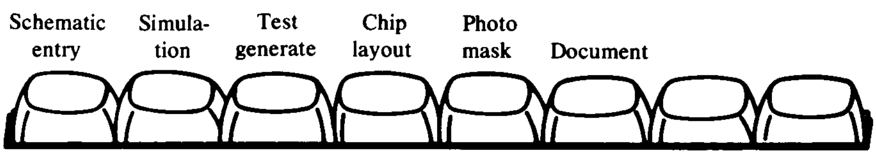
A very obvious set of choices under the digital analyzer key choice might be



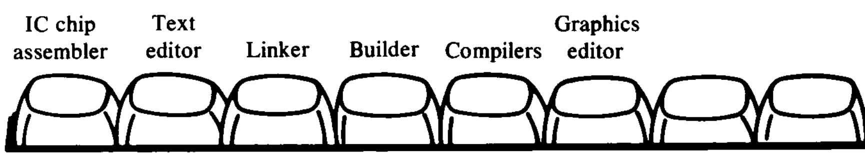
If PC (printed circuit) is pressed, the choices are



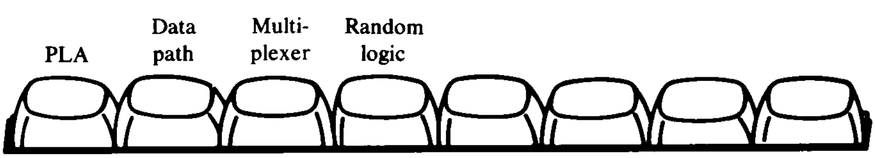
When IC (integrated circuit) is pressed, the choices are



When chip layout is pressed, the choices are [8]



When IC chip assembler is pressed, the choices are



The technology that allows us to consider the true possibility of such a system is based heavily on the VLSI extensions that the system intends to support. For example, the reduction of major equipment such as a sophisticated logic state analyzer to a one- or two-card module allows zoom potential because several different modules can be resident in the card cage of a work station. Also, a cluster network can be composed of different configurations in each work station and, potentially, could even include a desktop computer for information graphics or management information systems.

Obviously, it is serving these needs for zoom that led to the reliance on the workhorse silicon efforts described in this chapter. It follows, however, that by using Ethernet linking of major data files and by clustering functional areas of concentrated computational power, it should be conceptually possible to link these relatively modest ideas about an electronic bench to the very sophisticated centers that will exist for state-of-the-art silicon progress, which may mean in the end that we all shall have the best of both worlds. To be sure, we need to be cognizant of the concerns of many frustrated users today, but such a requirement should be paramount in our design perspective.

The next few years should see the significant development of tools to enable the electronic bench concept to be realized. This electronic bench will encompass the necessary synthesis, analysis, and linking functions. Clearly, the cost of such a powerful automated design center will be dramatically reduced, concurrent with substantially improved combinational performance. With the aid of such instrumentation concepts, we hope to support the design and analysis of the VLSI era.

ACKNOWLEDGMENTS

The authors would like to thank those who contributed to this work, in particular: Merrill Brooksby, Manager of the Corporate CAD Group; John Strathman, Larry McBride, and Kyle Black of the Colorado Springs Division Technology Center; and Ted Hanna, Rick Nygaard, Rick Palmer, Joel Zellmer, and Jeff Sondeen of the Logic Systems Operation.

REFERENCES

1. H. Bogart, Dataquest, *Semi Inform. Service Seminar, Newport Beach, California, January 28–30* pp. 205–206, 1980.
2. J. Beyers, *et al.*, A 32b VLSI processing system, *Proc. Int. Solid-State Circuits Conf., February 18–20* (1981).
3. F. P. Brooks Jr., "The Mythical Man-Month." Addison-Wesley, Reading, Massachusetts, 1978.
4. C. Clare, "Designing Logic Systems Using State Machines." McGraw-Hill, New York, 1973.
5. C. Mead and L. Conway, "Introduction to VLSI Systems." Addison-Wesley, Reading, Massachusetts, 1980.
6. L. M. Rosenberg, The evolution of design automation to meet the challenges of VLSI, *Design Automat. Proc., 17th* ACM/SIGDA and IEEE Computer Society, 1980.

7. T. W. Sidle, Weakness of commercial data base management systems in engineering applications, *Design Automat. Proc., 17th* ACM/SIGDA and IEEE Computer Society, 1980.
8. D. Johannsen, Bristle blocks: A silicon compiler, *Proc. CalTech Conf. VLSI, Pasedena, California* (January 1979).

ADDITIONAL REFERENCES

H. J. DeMan, Computer-aided design for integrated circuits: Trying to bridge the gap, *IEEE J. Solid State Circuits* **SC-14** (3) 613–621 (1979).

R. G. Greer, Computer aided coarse grid layout techniques for photomasks, *Solid State Technol.* 65–69 (September 1979).

D. Gibson and S. Nance, June 1979, SLIC: Symbolic layout of integrated circuits, *Proc. 13th Design Automation Conf. San Francisco, California, June 1979.* pp. 434–440.

J. D. Williams, STICKS—A graphical compiler for high-level LSI design, *AFIPS Conf. Proc.* **47,** 289–295 (June 1978).